Bibliotech — Professor Sequence Diagrams

Project: Bibliotech
Document: Runtime sequences for the main professor actions
Generated: 2026-10-27
Scope: Professor login/dashboard, publishing material, reviewing student suggestions, associating existing material, editing material, managing observers, removing material from a subject, and the API-only subject-association request.

All API calls made by the frontend use `apiService`, which adds `Authorization: Bearer <JWT>` when a session exists. Protected routes also run `verifyToken` and `requireRole`. The diagrams omit that header on each arrow to keep them readable.

1. Action Map

Professor action	UI entry point	API route	Main service method	Audit label
Log in and open dashboard	`/login` → `/profesor`	`POST /api/auth/login`, `GET /api/subjects`, `GET /api/materials/pending-review`, `GET /api/materials/mine`	`AuthService.login`, `SubjectService.getAllSubjects`, `MaterialService.getPendingForProfessor`, `getMyUploads`	none on login
Publish new material	Teacher dashboard form	`POST /api/materials/upload`	`MaterialService.uploadOrSuggestMaterial`	`Subida Material` (new) / `Asociacion Material` (duplicate)
Review student suggestions	Pending review panel	`GET /api/materials/pending-review`, then `POST /api/materials/:id/approve` or `/reject`	`getPendingForProfessor`, `approveMaterial`, `rejectMaterial`	`Aprobacion Material` / `Cancelacion Material`
Add existing material to own subjects	Material card → "Add to subjects" modal	`POST /api/materials/:id/associate`	`MaterialService.associateMaterialToSubjects`	`Asociacion Material`
Edit material	My uploads / subject page → editor	`PUT /api/materials/:id`	`MaterialService.update`	`Modificacion Material`
View observers and ban a student	"Observers" on a subject	`GET /api/subscriptions/subject/:subjectId/observers`, `POST /api/subscriptions/ban/:alumnoId/:subjectId`	`SubscriptionService.getObservers`, `banStudent`	`Baneo Alumno`
Remove material from one subject	Subject detail → remove	`DELETE /api/materials/:id/subjects/:subjectId`	`MaterialService.delete(..., subjectId)`	`Desasociacion Material`
Request a subject association	API only; no current page control	`POST /api/subjects/:id/request-association`	`SubjectService.requestAssociation`	`Solicitud Materia`
Confirm a subject association	Admin dashboard (the professor waits)	`POST /api/subjects/:id/confirm-association`	`SubjectService.confirmAssociation`	`Asociacion Materia`

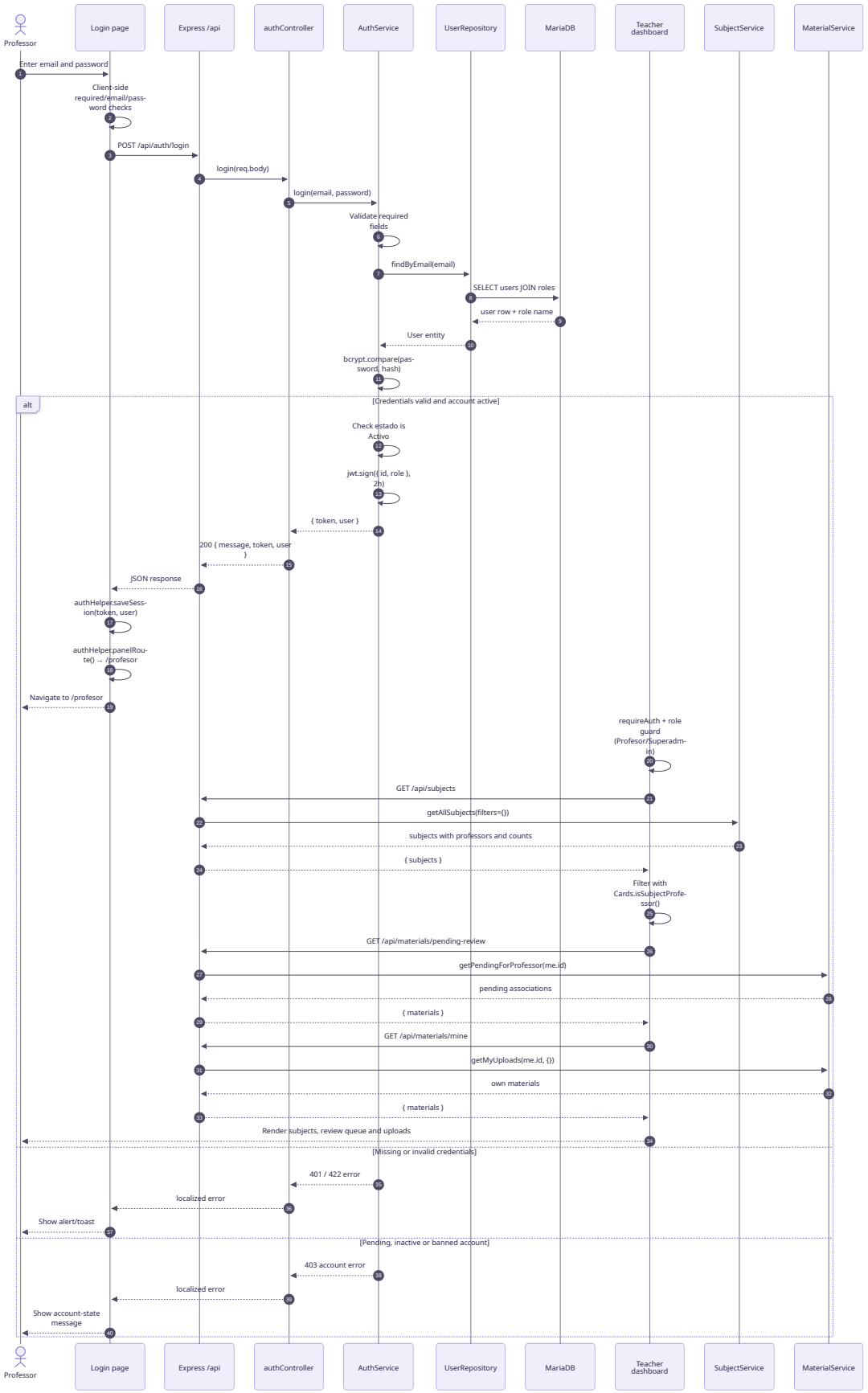
Professor-only preconditions

- The professor must have been `Activo` when the JWT was issued. `verifyToken` does not re-check account state, so a later ban or soft delete does not invalidate an existing token before expiry.
- The professor must be assigned to the subject through `subject_professors`.
- `subjects.profesor_id` alone is not sufficient; `Subject.hasProfessor()` uses `subject_professors` with a fallback to `profesor_id`.
- A fresh database has 23 subjects but **no professor assignments**, so a professor dashboard will be empty until an administrator assigns subjects.

2. Login and Dashboard Initialization

Preconditions

- A professor account exists and is `Activo`.
- The account has a password hash.
- The professor is assigned to at least one subject (otherwise the dashboard loads empty).
- The browser has no valid session, or the user intentionally logs in again.



Postconditions

- The JWT is stored by `authHelper` with the safe user object.

- The dashboard has loaded subjects, pending review rows and the professor's uploads.
- Login itself does **not** create an audit row.
- If the professor has no assignments, `mySubjects` is empty and the dashboard shows the "no subjects" state.

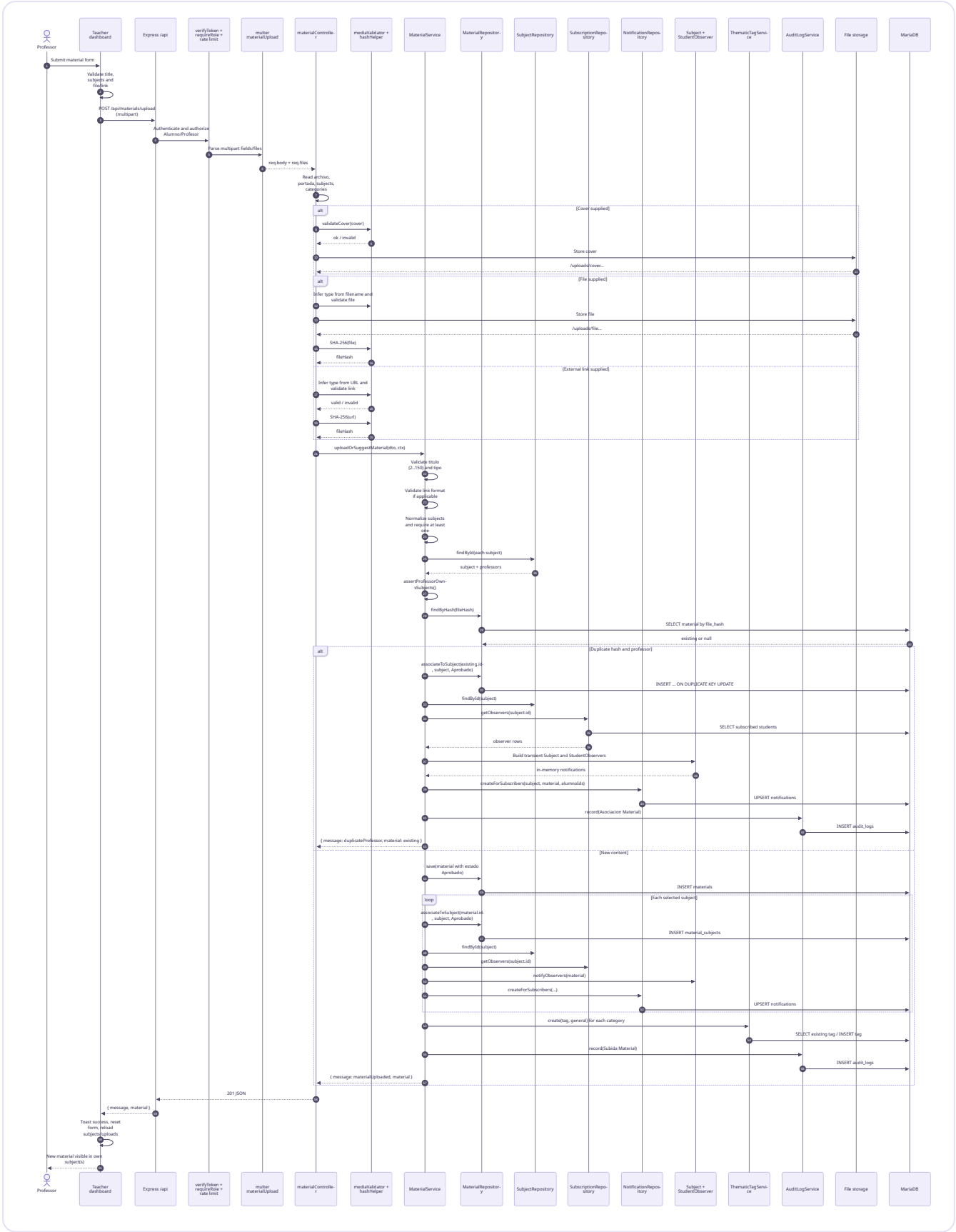
Failure notes

- `GET /api/subjects` is public/optional-auth; the dashboard filters client-side.
- `GET /api/materials/pending-review` is restricted to `Profesor` / `Superadmin`. For a `Superadmin`, the repository still filters by `subject_professors.profesor_id = currentUserId`, so an unassigned admin normally sees an empty list.
- Any later 401 from `apiService` clears the session and redirects to `/login`.

3. Publish New Material

Preconditions

- Professor is authenticated and assigned to at least one subject.
- The form contains a title, a type, one or more subject IDs, and either one file or one external URL.
- A cover image is optional.
- The file type is consistent with the selected type.


Professor
Teacher dashboard
Express /api
verifyToken + requireRole + rate limit
multer materialUpload
materialControlle-r
mediaValidator + hashHelper
MaterialService
MaterialRepositor-y
SubjectRepository
SubscriptionRepo-sitory
NotificationRepos-itory
Subject + StudentObserver
ThematicTagServi-ce
AuditLogService
File storage
MariaDB
Submit material form
Validate title, subjects and fileLink
POST /api/materials/upload (multipart)
Authenticate and authorize Alumno/Profesor
Parse multipart fields/files
req.body + req.files
Read archivo, portada, subjects, categories
alt
[Cover supplied]
validateCover(cover)
ok / invalid
Store cover
/uploads/cover...
[File supplied]
Infer type from filename and validate file
Store file
/uploads/file...
SHA-256(file)
fileHash
[External link supplied]
Infer type from URL and validate link
valid / invalid
SHA-256(url)
fileHash
uploadOrSuggestMaterial(dto, ctx)
Validate título (2..150) and tipo
Validate link format if applicable
Normalize subjects and require at least one
findById(each subject)
subject + professors
assertProfessorOwn-sSubjects()
findByHash(fileHash)
SELECT material by file_hash
existing or null
[Duplicate hash and professor]
associateToSubject(existing.id-, subject, Aprobado)
INSERT ... ON DUPLICATE KEY UPDATE
findById(subject)
getObservers(subject.id)
SELECT subscribed students
observer rows
Build transient Subject and StudentObservers
in-memory notifications
createForSubscribers(subject, material, alumnoIds)
UPSERT notifications
record(Asociacion Material)
INSERT audit_logs
{ message: duplicateProfessor, material: existing }
[New content]
save(material with estado Aprobado)
INSERT materials
[Each selected subject]
loop
associateToSubject(material.id-, subject, Aprobado)
INSERT material_subjects
findById(subject)
getObservers(subject.id)
notifyObservers(material)
createForSubscribers(...)
UPSERT notifications
create(tag, general) for each category
SELECT existing tag / INSERT tag
record(Subida Material)
INSERT audit_logs
{ message: materialUploaded, material }
201 JSON
{ message, material }
Toast success, reset form, reload subjects/uploads
New material visible in own subject(s)

Alternative and error paths

Condition	Result	Cleanup
Missing title, type, subject, or file/link	`422` validation error	Controller deletes any files already written in local mode.
File extension/MIME does not match `tipo`	`422 validation.fileType`	Same.
Invalid cover	`422 validation.coverType`	Same.
Invalid YouTube/Drive/repository link	`422 validation.invalidLink`	Same.
Professor does not teach one of the selected subjects	`403`	Same.
Duplicate title/file from another author	Not blocked by title; duplicate **file hash** follows the duplicate flow.	Existing material is associated; the newly uploaded duplicate file/cover remains in storage because the controller cleans up only on errors (and only local files are deleted).
Too many upload requests	`429` from `uploadRateLimit`	No service call.
Thematic tag is banned	`ThematicTagService.create()` fails; the error is ignored so the material still publishes.	—
Notification persistence fails	Logged by `MaterialService`; upload still succeeds.	Notification may be missing.

Postconditions

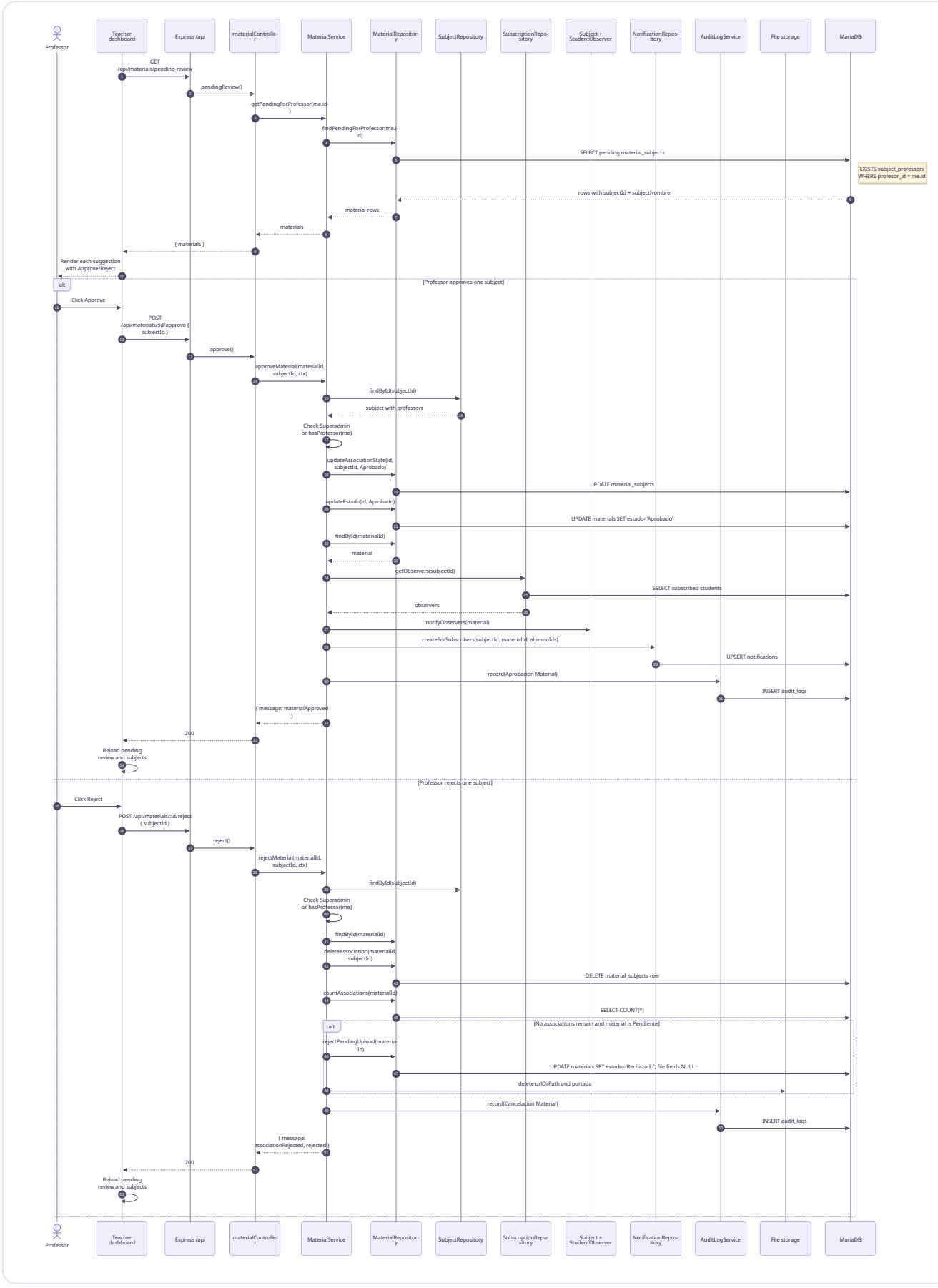
- `materials` row exists with `estado = 'Aprobado'`.
- One `material_subjects` row per selected subject with `estado_aprobacion = 'Aprobado'`.
- Subscribers receive durable notification rows.
- Audit row `Subida Material` (new) or `Asociacion Material` (duplicate).
- The file/cover is stored in the configured storage backend.
- **Duplicate caveat:** the duplicate branch only upserts the association. It does **not** change the existing material's global `estado`; if the existing material is still `Pendiente`, the professor's approved association alone does not make it visible on subject pages.

4. Review Pending Student Suggestions

Preconditions

- A `material_subjects` row is in `estado_aprobacion = 'Pendiente'`.

- The material is usually `Pendiente`, but it can already be `Aprobado` when a student suggested a duplicate of an approved resource.
- The professor is assigned to the subject through `subject_professors`.
- The professor is authenticated.


Professor
Teacher dashboard
Express /api
materialControlle-r
MaterialService
MaterialRepositor-y
SubjectRepository
SubscriptionRepo-sitory
Subject + StudentObserver
NotificationRepos-itory
AuditLogService
File storage
MariaDB
GET /api/materials/pending-review
pendingReview()
getPendingForProfessor(me.id)
findPendingForProfessor(me.id)
SELECT pending material_subjects
EXISTS subject_professors WHERE profesor_id = me.id
rows with subjectId + subjectNombre
material rows
materials
{ materials }
Render each suggestion with Approve/Reject
alt
[Professor approves one subject]
Click Approve
POST /api/materials/:id/approve { subjectId }
approve()
approveMaterial(materialId, subjectId, ctx)
findById(subjectId)
subject with professors
Check Superadmin or hasProfessor(me)
updateAssociationState(id, subjectId, Aprobado)
UPDATE material_subjects
updateEstado(id, Aprobado)
UPDATE materials SET estado='Aprobado'
findById(materialId)
material
getObservers(subjectId)
SELECT subscribed students
observers
notifyObservers(material)
createForSubscribers(subjectId, materialId, alumnoIds)
UPSERT notifications
record(Aprobacion Material)
INSERT audit_logs
{ message: materialApproved }
200
Reload pending review and subjects
[Professor rejects one subject]
Click Reject
POST /api/materials/:id/reject { subjectId }
reject()
rejectMaterial(materialId, subjectId, ctx)
findById(subjectId)
Check Superadmin or hasProfessor(me)
findById(materialId)
deleteAssociation(materialId, subjectId)
DELETE material_subjects row
countAssociations(materialId)
SELECT COUNT(*)
alt
[No associations remain and material is Pendiente]
rejectPendingUpload(materialId)
UPDATE materials SET estado='Rechazado', file fields NULL
delete urlOrPath and portada
record(Cancelacion Material)
INSERT audit_logs
{ message: associationRejected, rejected }
200
Reload pending review and subjects

Alternative and error paths

Condition	Result
Subject does not exist	`404`
Professor does not teach the subject and is not Superadmin	`403`
Material does not exist (reject path)	`404`
Approve is called for an association that no longer exists	Current code updates/creates no association but still marks the material `Aprobado`; the UI normally prevents this.
Rejection is the last association of a pending material	Material becomes `Rechazado`; files are deleted; `rejected=true`.
Rejection leaves other associations	Material remains visible for those subjects; `rejected=false`.
Rejection removes the last association but material is already `Aprobado`	Association is deleted; material stays `Aprobado` with no subject associations, files are kept, and `rejected=false`.
Notification persistence fails	Logged; approval/rejection still succeeds.

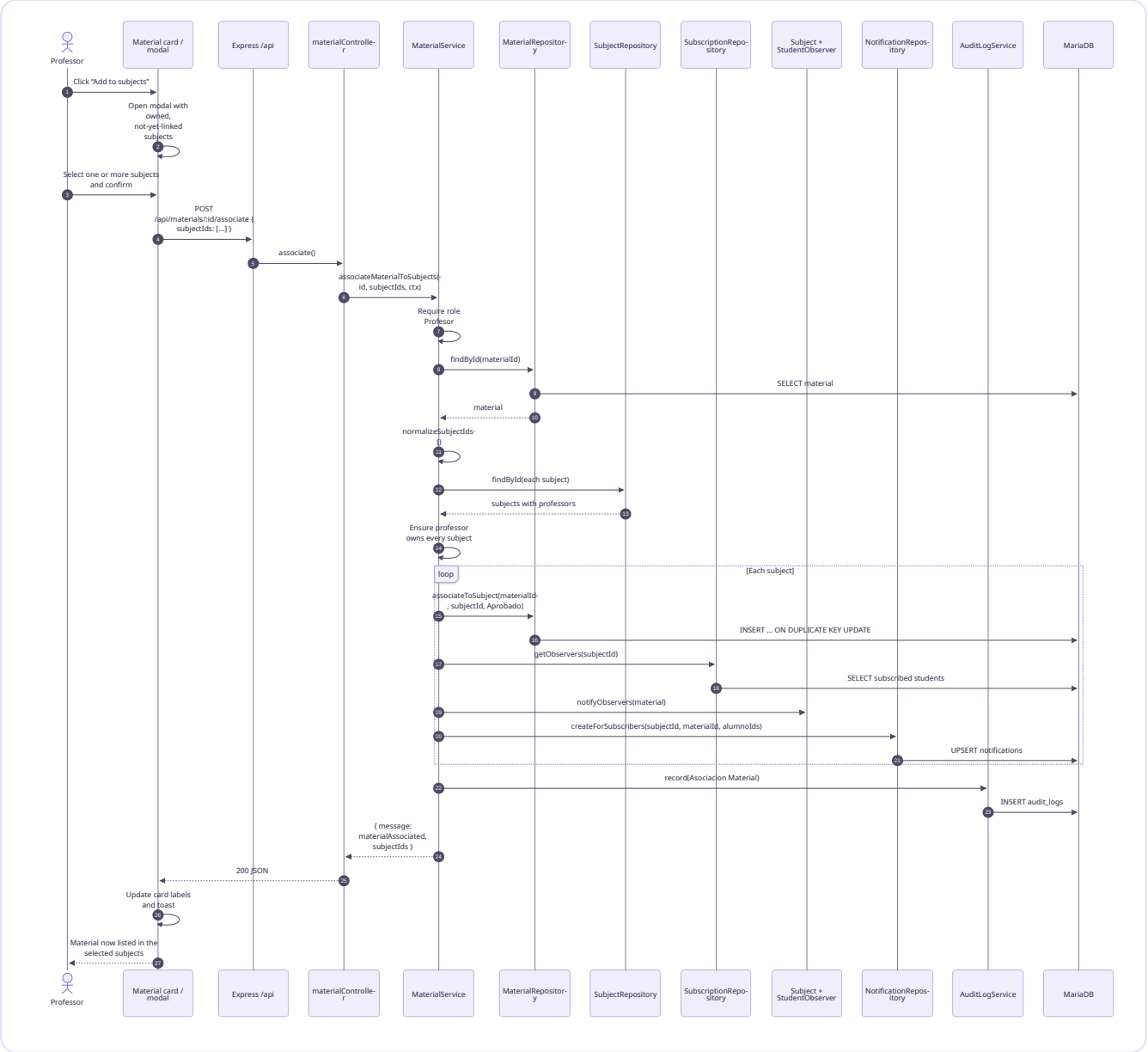
Postconditions

- Approve: association `Aprobado`; material `Aprobado`; subscribers notified; audit `Aprobacion Material`.
- Reject: association deleted; if it was the last one and the material was pending, material `Rechazado` and files removed; audit `Cancelacion Material`.
- The pending-review query returns only associations that are still pending and belong to the professor's subjects; it does not require the material itself to be `Pendiente`.

5. Add an Existing Material to Own Subjects

Preconditions

- The material is already visible to the professor (usually approved).
- The professor teaches the target subjects.
- The material is not already associated with those subjects (the UI hides existing ones, but the backend uses an upsert).



Alternative and error paths

Condition	Result
Material not found	`404`
Caller is not a professor	`403` (route also enforces this)
Missing/empty/invalid `subjectIds`	`422`
Any selected subject does not exist or is not taught by the caller	`403` (the service stops before associating)
Subject already linked	Upsert succeeds; count remains one row per pair.
Notification persistence fails	Logged; association still succeeds.

Postconditions

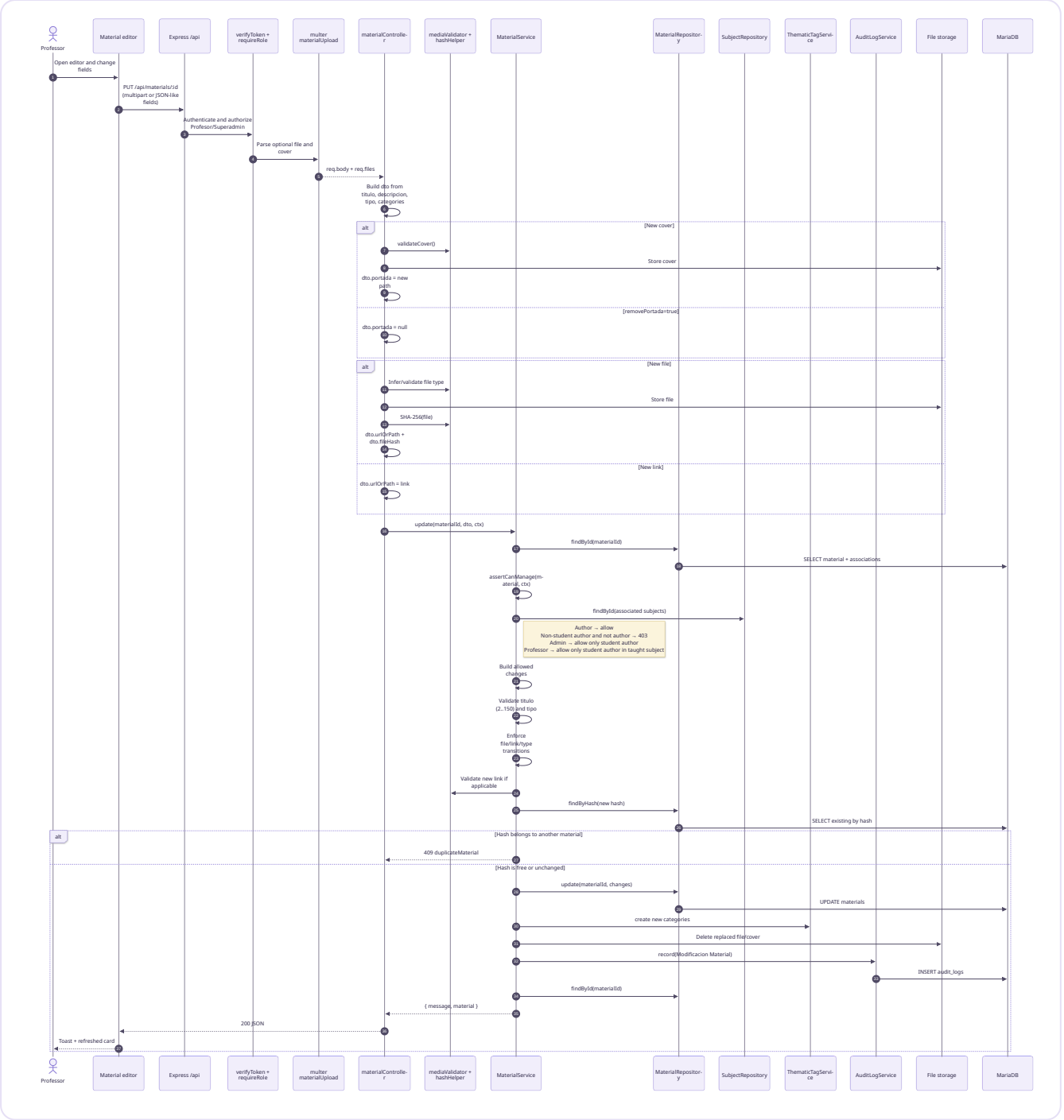
- New `material_subjects` rows with `Aprobado`.

- An existing `(material, subject)` pair is upserted to `estado_aprobacion = 'Aprobado'` rather than duplicated; other pairs are unchanged.
- Subscribers of each selected subject receive a notification, even when the `materials` row itself is still `Pendiente`.
- Audit `Asociacion Material`.
- The `materials` row itself is not modified or duplicated.
- If the existing material is still `Pendiente`, this endpoint adds an `Aprobado` association but does not change the material's global state; it remains invisible on subject pages until approved.

6. Edit a Material

Preconditions

- The material exists.
- The user is the author; a professor handling a student suggestion in a subject they teach; or a Superadmin handling a student's material. A professor cannot edit another professor's material even if they teach the same subject.
- The frontend shows the edit action only when `Cards.canManage` allows it; the backend re-checks.



Editable fields and rules

Field	Rule
`titulo`	Trimmed; required if supplied; 2–150 characters.
`descripcion`	Trimmed; empty becomes `NULL`.
`categories`	Array or comma-separated input; trimmed; case-insensitive deduplication. New tags are created in the catalog.
`tipo`	Must be one of `PDF`, `EPUB`, `Word`, `Presentacion`, `Link_Drive`, `Youtube`, `Repositorio`.

Field	Rule
`archivo`	Only for file types; required when changing from a link to a file type or when changing file type.
`urlOrPath`	Required for link types; validated by type.
`portada`	Optional image; `removePortada=true` clears it.
`fileHash`	Derived by the controller; not accepted directly from the client.

Permission matrix

Caller	Target	Result
Author (Professor/Superadmin)	Own material	Allowed
Author (Student)	Own material	Route rejects with `403`; student authors use cancel instead
Professor	Another professor's material	`403`
Professor	Student suggestion in a subject they teach	Allowed
Professor	Student suggestion in a subject they do not teach	`403`
Superadmin	Student material	Allowed
Superadmin	Another professor's/admin's material	`403`: `assertCanManage` rejects any material whose author is not an `Alumno` before checking the admin role. Only the author can edit it.
Student	Any material	Route rejects with `403`; students use cancel, not edit.

Alternative and error paths

Condition	Result
Material not found	`404`
No actual changes	`422 invalidData`
Invalid type transition	`422 fileType` / `invalidData`
New file hash already used by another material	`409 duplicateMaterial`
Invalid link	`422 invalidLink`
Cover invalid	`422 coverType`
Old file/cover deletion fails	Logged; update still succeeds.

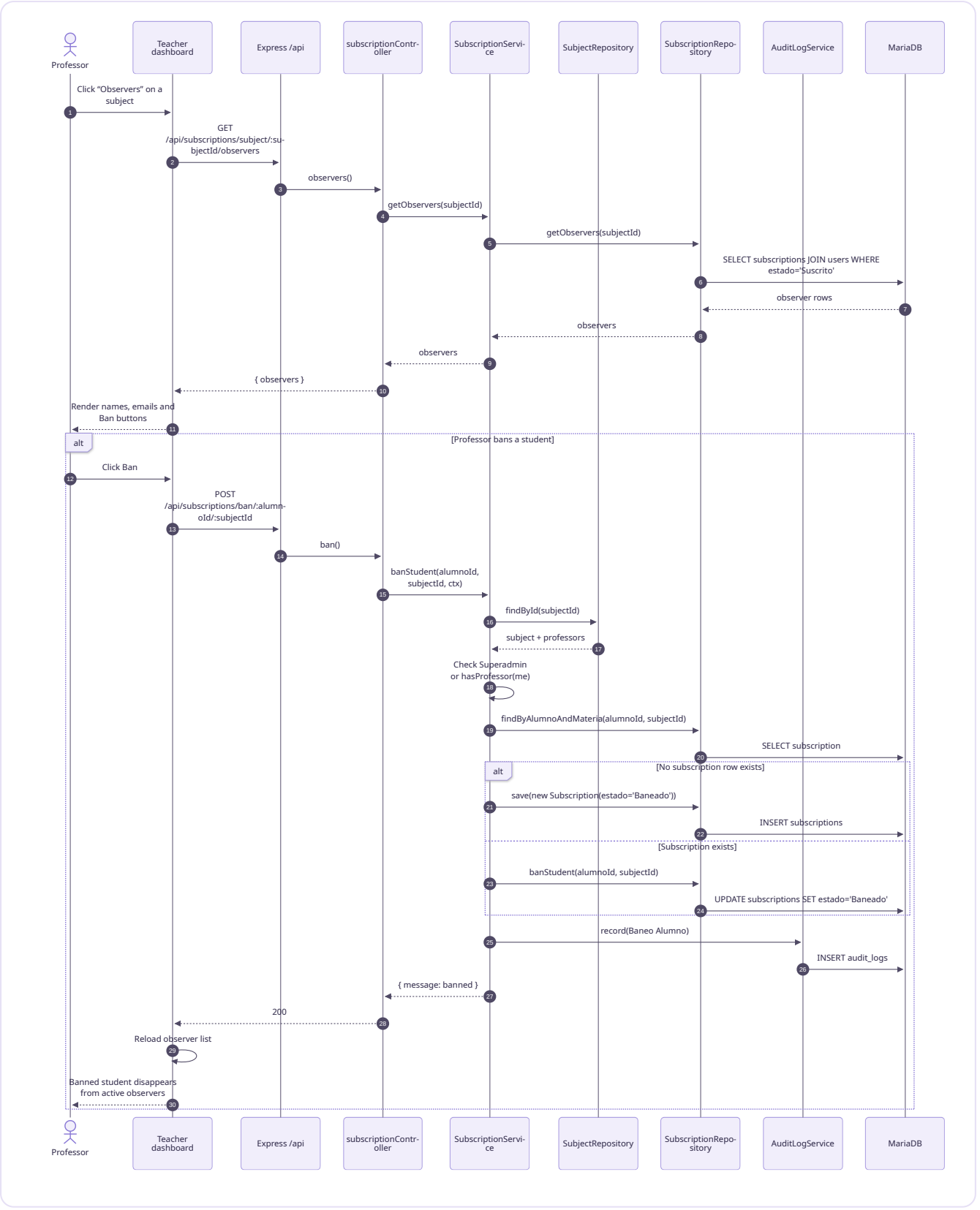
Postconditions

- Updated `materials` row.
- Replaced files removed from storage where possible.
- New tags created if needed.
- Audit `Modificacion Material`.

7. View Observers and Ban a Student

Preconditions

- The professor is authenticated.
- For viewing: any Professor/Admin can call the route; the service does not verify subject ownership.
- For banning: the caller must teach the subject or be Superadmin.



Alternative and error paths

Condition	Result
Subject not found	`404`

Condition	Result
Professor does not teach the subject (and is not Superadmin)	`403`
Student has no subscription row	A `Baneado` row is created; the student cannot subscribe.
Student was already subscribed	Row is updated to `Baneado`.
Banned student calls subscribe	`403 errors.forbidden`
Viewing observers for a subject the professor does not teach	**Allowed** by the current service; this is a privacy gap to consider.

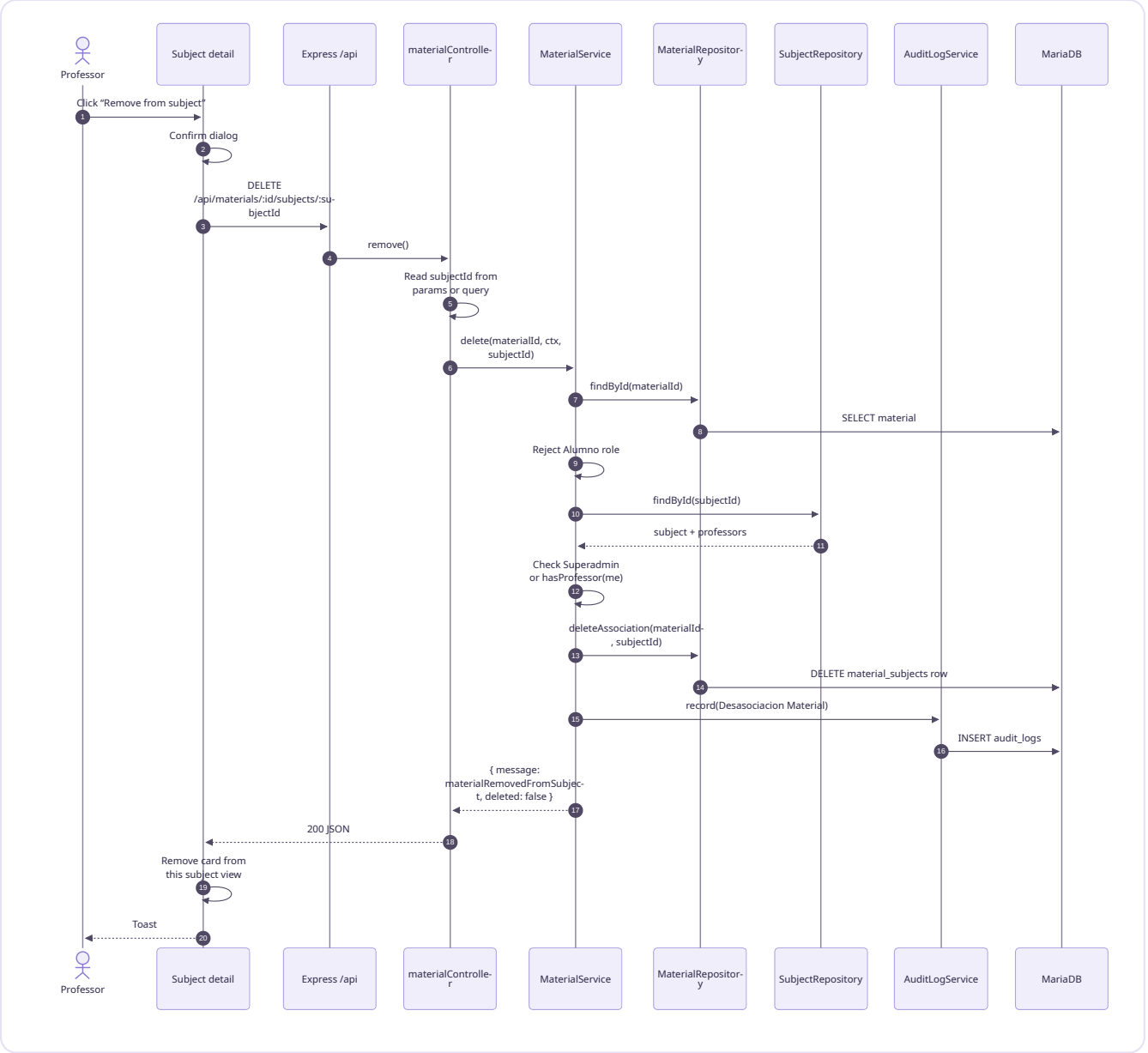
Postconditions

- Subscription state is `Baneado`.
- Banned student no longer appears in `getObservers` (which filters `Suscrito`).
- Student cannot re-subscribe while the banned row exists.
- Audit `Baneo Alumno` points to the student user id, not the subject.

8. Remove Material from One Subject

Preconditions

- Material is associated with the subject.
- The caller is a professor of that subject or a Superadmin.
- The route is the subject-specific delete, not the global admin delete.



Alternative and error paths

Condition	Result
Material not found	`404`
Subject not found	`404`
Caller is not a professor of that subject and not Superadmin	`403`
Caller is a student	Route rejects with `403`
Association does not exist	Delete affects 0 rows but still returns success and writes an audit row.

Postconditions

- Only the `(material_id, subject_id)` association is removed.
- The material remains in the global catalog and in its other subjects.

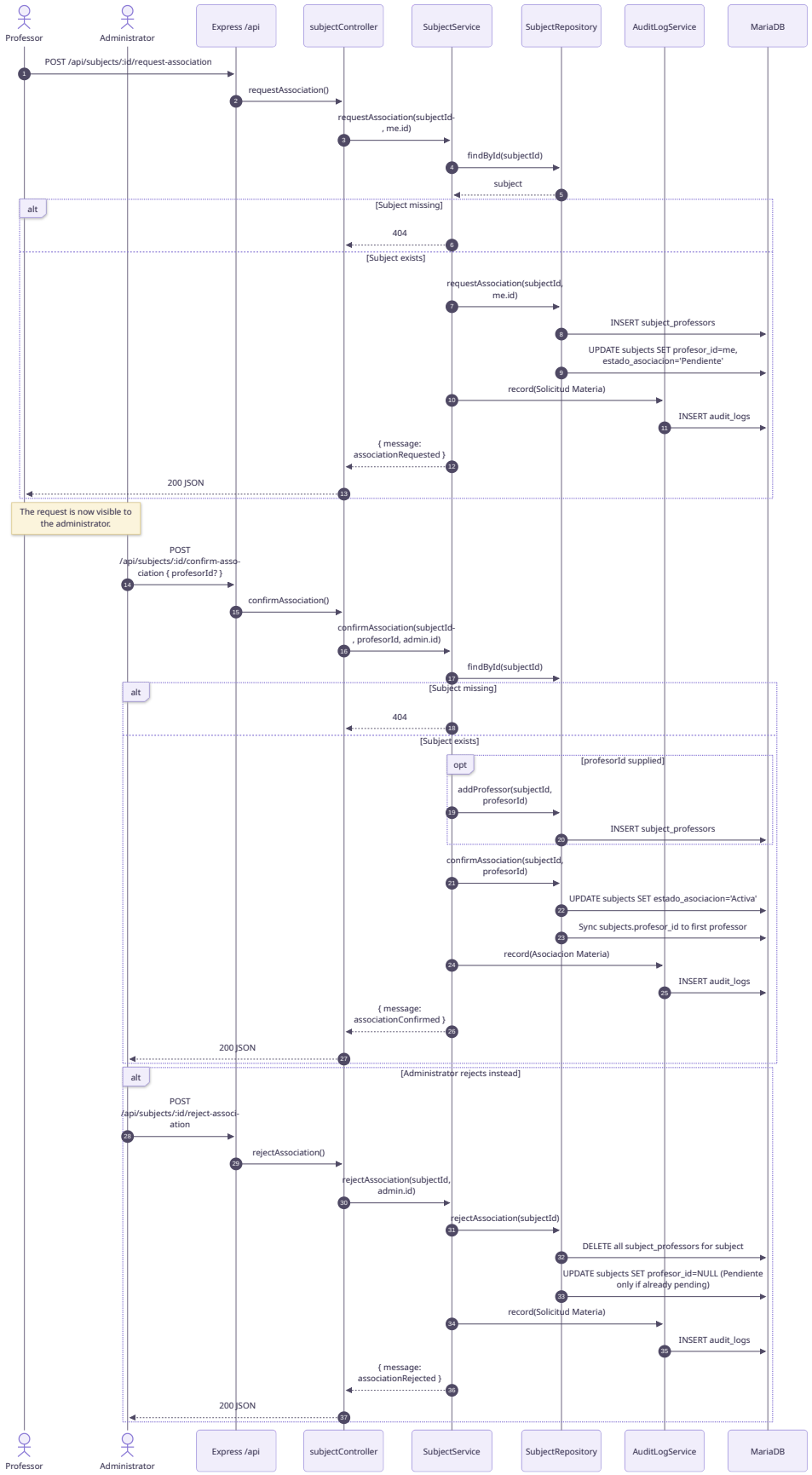
- The material is not deleted and its files are not removed.
- Audit `Desasociacion Material`.
- This differs from rejection, which may reject and clear a pending material.

9. Request Subject Association and Admin Confirmation

This flow exists in the API and tests, but the current frontend has **no professor-facing control** for requesting an association. It is included because it is a professor action defined by the domain.

Preconditions

- A subject exists (optionally without the professor, for example after admin creation).
- The professor is authenticated.
- The administrator reviews the request.


Professor
Administrator
Express /api
subjectController
SubjectService
SubjectRepository
AuditLogService
MariaDB
POST /api/subjects/:id/request-association
requestAssociation()
requestAssociation(subjectId-, me.id)
findById(subjectId)
subject
alt
[Subject missing]
404
[Subject exists]
requestAssociation(subjectId, me.id)
INSERT subject_professors
UPDATE subjects SET profesor_id=me, estado_asociacion='Pendiente'
record(Solicitud Materia)
INSERT audit_logs
{ message: associationRequested }
200 JSON
The request is now visible to the administrator.
POST /api/subjects/:id/confirm-asso-ciation { profesorId? }
confirmAssociation()
confirmAssociation(subjectId-, profesorId, admin.id)
findById(subjectId)
[Subject missing]
404
[Subject exists]
opt
[profesorId supplied]
addProfessor(subjectId, profesorId)
INSERT subject_professors
confirmAssociation(subjectId, profesorId)
UPDATE subjects SET estado_asociacion='Activa'
Sync subjects.profesor_id to first professor
record(Asociacion Materia)
INSERT audit_logs
{ message: associationConfirmed }
200 JSON
[Administrator rejects instead]
POST /api/subjects/:id/reject-associ-ation
rejectAssociation()
rejectAssociation(subjectId, admin.id)
rejectAssociation(subjectId)
DELETE all subject_professors for subject
UPDATE subjects SET profesor_id=NULL (Pendiente only if already pending)
record(Solicitud Materia)
INSERT audit_logs
{ message: associationRejected }
200 JSON

Alternative and error paths

Condition	Result
Subject not found	`404`
Requesting user is not a professor	Route rejects with `403`
Request is duplicated	The `subject_professors` upsert keeps one row; the subject remains pending.
Admin confirms without `profesorId`	Existing assignment is kept and the subject becomes active.
Admin confirms with a valid non-professor `profesorId`	The id is inserted because this path does not re-check the `Profesor` role; only the user FK is enforced.
Admin rejects	All assignments removed. The subject becomes unassigned; it stays `Pendiente` only if it was already pending, otherwise it remains `Activa` with no professors.
Professor tries to request a subject that already has another professor	The current service adds the caller as an additional professor; there is no exclusivity rule.

Postconditions

- Request: `subject_professors` row, cached `profesor_id`, `estado_asociacion = 'Pendiente'`, audit `Solicitud Materia`.
- Confirm: subject `Activa`, cached professor synchronized, audit `Asociacion Materia`.
- Reject: no professors, `profesor_id = NULL`; `estado_asociacion` becomes `Pendiente` only if it was already pending, otherwise it remains `Activa`. Audit `Solicitud Materia`.

10. Cross-Cutting Notes

Topic	Behavior
Authentication	JWT expires after 2 hours; `apiService` clears the session on 401.
Authorization	Route role checks plus service ownership checks. Frontend role checks are convenience only.
Subject ownership	Based on `subject_professors`; `subjects.profesor_id` is a cached fallback.
Notifications	Created for `Suscrito` students only. The unique constraint upserts the same user/subject/material and resets `read_at`.
Audit	Write services record labels such as `Subida Material`, `Aprobacion Material`, `Modificacion Material`, `Asociacion Material`, `Desasociacion Material`, `Cancelacion Material`, `Baneo Alumno`.
File cleanup	On request failure, controllers delete newly written files only in `FILES_MODE=local`; cloud uploads are not removed by that fallback. Services clean up replaced/deleted files through the storage helper. Disk cleanup errors are logged, not fatal.

Topic	Behavior
Duplicate uploads	SHA-256 file hash is unique. A professor duplicate adds the existing material to their subject; a student duplicate creates a pending association. The duplicate file/cover uploaded in that request is left in storage (orphan) because cleanup runs only on errors in local mode.
Pending limit	Students may have at most 3 materials in `Pendiente` state.
Response messages	Services return translation keys; controllers/`ok()` localize them.
Client pagination	API list responses are not paginated, except favorites' internal `limit` / `offset`; the frontend paginates locally (default 6 items).
Professor UI scope	The teacher dashboard also allows `Superadmin`, but professor-specific data still depends on actual `subject_professors` assignments.